

Course

mitaoe.codetantra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d66/6684d40e5d95376955a73c1d

CODETANTRA

HomeLearn Anywhere

202501090110@mitaoe.ac.inSupportLogout

3.1.1. Numpy Array Operations01:14

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

numpyarr...

```
1 import numpy as np
2 r, c = map(int, input().split())
3 elements = []
4 for _ in range(r):
5     row = list(map(int, input().split()))
6     elements.extend(row)
7 arr = np.array(elements).reshape(r, c)
8 print(arr)
9 print(arr.ndim)
10 print(arr.shape)
11 print(arr.size)
12
```

TerminalTest cases

33°C Sunny

Search

11:39 08-05-2026

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d67/6774e3ff4ab787eca9da092

CODETANTRA

Home Learn Anywhere

202501090110@mitaoe.ac.in Support Logout

3.2.1. Numpy: Matrix Operations

09:25

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. Addition (`matrix_a + matrix_b`)
2. Subtraction (`matrix_a - matrix_b`)
3. Element-wise Multiplication (`matrix_a * matrix_b`)
4. Matrix Multiplication (`matrix_a · matrix_b`)
5. Transpose of Matrix A

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases

matrixOp...

1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10
11 # Addition
12 print("Addition (A + B):")
13 print(matrix_a + matrix_b)
14 # Subtraction
15 print("Subtraction (A - B):")
16 print(matrix_a - matrix_b)
17 # Multiplication (element-wise)
18 print("Element-wise Multiplication (A * B):")
19 print(matrix_a * matrix_b)
20 # Matrix multiplication (dot product)
21 print("A dot B:")
22 print(np.dot(matrix_a, matrix_b))
23 # Transpose
24 print("Transpose of A:")
25 print(matrix_a.T)

Terminal Test cases

33°C Sunny

Search

11:39 08-05-2026

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774ee19f4ab787eca9da7fe

CODETANTRA

Home Learn Anywhere

202501090110@mitaoe.ac.in Support Logout

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays01:32

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking** Stack the two matrices horizontally (side by side).
- Vertical Stacking** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 h_stack = np.hstack((arr1, arr2))
12 print("Horizontal Stack:")
13 print(h_stack)
14 # Perform vertical stacking (vstack)
15 v_stack = np.vstack((arr1, arr2))
16 print("Vertical Stack:")
17 print(v_stack)
```

Terminal Test cases

33°C Sunny

Search

11:39 08-05-2026

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774ee9df4ab787eca9da869

CODETANTRA

Home Learn Anywhere

202501090110@mitaoe.ac.in Support Logout

3.2.3. Numpy: Custom Sequence Generation00:57

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

The program should print the generated sequence based on the input values.

Sample Test Cases

customS...

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8
9 # Generate the sequence using np.arange()
10 sequence = np.arange(start, stop, step)
11 # Print the generated sequence
12 print(sequence)
```

Terminal Test cases

< Prev Reset Submit Next >

33°C Sunny

Search

11:39 08-05-2026

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5e/6774ef2cf4ab787eca9da8c0

CODETANTRA

Home Learn Anywhere

202501090110@mitaoe.ac.in Support Logout

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators00:51

You are given two arrays `a` and `b`. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

- Arithmetic Operations:**
 - Compute the element-wise sum, difference, and product of the two arrays.
- Statistical Operations:**
 - Calculate the mean, median, and standard deviation of array `a`.
- Bitwise Operations:**
 - Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: `A1` OR `B1`).

Input Format:

- The first line contains space-separated integers representing the elements of array `A`.
- The second line contains space-separated integers representing the elements of array `B`.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

different...

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     import numpy as np
6
7     def array_operations(A, B):
8
9         # Convert A and B to NumPy arrays
10        A = np.array(A)
11        B = np.array(B)
12
13        # Arithmetic Operations
14        sum_result = A + B
15        diff_result = A - B
16        prod_result = A * B
17
18        # Statistical Operations
19        mean_A = np.mean(A)
20        median_A = np.median(A)
21        std_dev_A = np.std(A)
22
23        # Bitwise Operations
24        and_result = A & B
25        or_result = A | B
26        xor_result = A ^ B
27
28        # Output results with one space between each element
29        print("Element-wise Sum:", ' '.join(map(str, sum_result)))
30        print("Element-wise Difference:", ' '.join(map(str, diff_result)))
31        print("Element-wise Product:", ' '.join(map(str, prod_result)))
32
33        print(f"Mean of A: {mean_A}")
34        print(f"Median of A: {median_A}")
35        print(f"Standard Deviation of A: {std_dev_A}")
36
```

Terminal Test cases

33°C Sunny

Search

11:39 08-05-2026

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774efaa4ab787eca9da91e

CODETANTRA

HomeLearn Anywhere

202501090110@mitaoe.ac.inSupportLogout

3.2.5. Numpy: Copying and Viewing Arrays01:49

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original_array and assigning it to view_array.
- Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: <original_array>
View array: <view_array>

- After modifying the copy:

Original array after modifying copy: <original_array>
Copy array: <copy_array>

Sample Test Cases

copyAnd...

```
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
12 copy_array = original_array.copy()
13
14
15 # Modify the view
16 view_array[0] = 99
17 print("Original array after modifying view:", original_array)
18 print("View array:", view_array)
19
20 # Modify the copy
21 copy_array[1] = 88
22 print("Original array after modifying copy:", original_array)
23 print("Copy array:", copy_array)
24
```

TerminalTest cases

Hot days ahead
35°C

Search

11:40
08-05-2026

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5e/698c54860aba96214c8b7d67/6774f064f4ab787eca9da9b0

CODETANTRA

Home Learn Anywhere

202501090110@mitaoe.ac.in Support Logout

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting01:15

The given code in the editor takes a single array, array1, as space-separated integers as input from the user. Additionally, it takes the following inputs:

- search_value: The value to search for in the array
- count_value: The value to count its occurrences in the array
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- Searching:** Find the indices where search_value appears in array1 and print these indices.
- Counting:** Count how many times count_value appears in array1 and print the count.
- Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
- Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

- A single line containing space-separated integers representing array1.
- An integer search_value represents the value to search for in the array.
- An integer count_value represents the value to count in the array.
- An integer broadcast_value represents the value to add to each element of the array.

Output Format:

- The indices where search_value occurs in array1.
- The count of occurrences of count_value in array1.
- The array after adding the broadcast_value to each element.
- The sorted array.

Sample Test Cases

arrayOpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11
12 # Find indices where value matches in array1
13 indices = np.where(array1 == search_value)[0]
14 print(indices)
15 # Count occurrences in array1
16 count = np.count_nonzero(array1 == count_value)
17 print(count)
18 # Broadcasting addition
19 broadcast_result = array1 + broadcast_value
20 print(broadcast_result)
21 # Sort the first array
22 sorted_array = np.sort(array1)
23 print(sorted_array)
```

Terminal Test cases

< Prev Reset Submit Next >

Hot days ahead 35°C

Search

ENG IN 11:40 08-05-2026

Course

mitaoe.codetantra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/67d7fb4f1d1a74a4035ef4225

CODETANTRA

Home Learn Anywhere

202501090110@mitaoe.ac.in Support Logout

3.2.7. Student Data Analysis and Operations01:35

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored >=90 in each subject:** Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored >=90:** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Operation...

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5 # 1. Print all student details
6 print("All student Details:\n",a)
7
8 # 2. print total students
9
10 print("Total Students:", len(a))
11
12 # 3. Print all student Roll numbers
13 print("All Student Roll Nos", a[:,0] ...)
14
15 # 4. Print subject 1 marks
16 print("Subject 1 Marks", a[:,1])
17
18 # 5. print minimum marks of Subject 2
19 print("Min marks in Subject 2", a[:,2].min() ...)
20
21 # 6. print maximum marks of Subject 3
22 print("Max marks in Subject 3", a[:,3].max() ...)
23
24 # 7. Print All subject marks
25 a1 = np.delete(a,0,1)
26 print("All subject marks:", a1.....)
27
28 # 8. print Total marks of students
29 print("Total Marks", a[:,1]+a[:,2]+a[:,3].....)
30
31 # 9. print average marks of each student
32 a2 = np.array(a[:,1]+a[:,2]+a[:,3])
33 A =np.around(a2/3,1)
34 print(A)
35
```

Terminal Test cases

Hot days ahead35°C

Search

ENG IN

11:4008-05-2026